

LAB 10: Constraint Satisfaction Problem: Formalization & Systematic Search

Objective

To rigorously formalize CSPs and implement uninformed and look-ahead based search strategies for computing consistent assignments under binary constraints.

Submission Guidelines:

1. Each task must properly show the output. DO NOT Clear the outputs after running.
2. **Download the .ipynb file** from Jupyter Notebook.
3. **Upload the .ipynb** to GCR.
4. **Failure to follow the submission guidelines will result in a deduction of 50% of the obtained marks.**

Task 1: Formal CSP Modeling of Map Coloring Problem

Construct a complete computational model of a CSP corresponding to the **Australia Map Coloring Problem**.

Requirements:

- Define a set of variables representing regions:
$$X = \{WA, NT, SA, Q, NSW, V, T\}$$
- Assign a uniform domain $D = \{Red, Green, Blue\}$
- Encode all **binary inequality constraints** between adjacent regions
- Represent constraints using:
 - Adjacency list **AND**
 - Constraint graph (using suitable data structure)

Additional Constraints:

- Your implementation must explicitly distinguish between:
 - Variables
 - Domains
 - Constraints (scope and relation)

Expected Output:

- Structured printout of CSP components
- Visualization (optional but encouraged)

Task 2: Backtracking Search for N-Queens Problem

Implement a **depth-first backtracking search algorithm** to solve the **N-Queens problem (N=8)**.

Requirements:

- Represent each queen as a variable X_i
- Domain: column indices for each row
- Constraints:
 - No two queens share same column
 - No two queens lie on same diagonal
- Implement:
 - Recursive backtracking procedure
 - Constraint checking before assignment

Additional Requirements:

- Ensure that:
 - Only one variable is assigned at each recursion depth
 - Search terminates upon first valid solution

Expected Output:

- Valid board configuration (8×8 matrix or coordinate list)

Task 3: Execution Trace of Backtracking Process

Extend Task 2 independently by implementing a **detailed execution trace generator** for backtracking.

Requirements:

- For each recursive call, log:
 - Current partial assignment
 - Variable selected
 - Value attempted
- Log all:
 - Constraint violations

- Backtracking events

Expected Output:

- Step-by-step trace demonstrating:
 - Depth-first traversal
 - Failure propagation
 - Backtracking points

Task 4: Forward Checking on Map Coloring CSP

Develop a CSP solver for the Map Coloring Problem incorporating **forward checking**.

Requirements:

- Maintain dynamic domains for all unassigned variables
- After each assignment:
 - Remove inconsistent values from neighboring domains
- Detect failure when:
 - Any domain becomes empty

Additional Requirements:

- Do NOT reuse previous implementations
- Implement forward checking as a standalone solver

Expected Output:

- Valid coloring assignment
- Domain reduction logs at each step

Task 5: Comparative Evaluation of Search Strategies

Independently implement two CSP solvers for the **4-Queens Problem**:

1. Basic Backtracking
2. Backtracking with Forward Checking

Requirements:

- Measure:
 - Number of recursive calls

- Number of constraint checks
- Number of backtracks
- Execute both solvers on identical input

Expected Output:

Strategy	Recursive Calls	Constraint Checks	Backtracks
Backtracking	XX	XX	XX
Backtracking + FC	XX	XX	XX

Task 6: University Course Scheduling using CSP

A university department needs to schedule **courses into time slots** such that no conflicts occur among instructors and student groups.

Requirements:

You are required to model and solve this as a CSP.

- Define variables:
 - Each course C_i represents a variable
- Domains:
 - Available time slots: {T1, T2, T3, T4, T5}
- Constraints:
 - No instructor teaches two courses at the same time
 - Courses with overlapping student groups must not share time slots
 - Certain courses are restricted to specific time slots (unary constraints)

Implementation Requirements:

- Implement a **backtracking-based CSP solver**
- Perform constraint checking before assignment
- Ensure:
 - Complete and consistent schedule is generated

Expected Output:

- A valid timetable:

C1 → T2

C2 → T4

C3 → T1

...

- Clearly indicate:
 - Constraint satisfaction verification

Task 7: Meeting Room Allocation with Forward Checking

A corporate office must assign **meeting rooms to scheduled meetings** while respecting timing and capacity constraints.

Requirements:

Model the problem as a CSP.

- Variables:
 - Each meeting M_i
- Domains:
 - Available rooms: {R1, R2, R3}
- Constraints:
 - No two meetings with overlapping time intervals can be assigned the same room
 - Each room has a capacity limit; meetings exceeding capacity cannot be assigned that room
 - Certain meetings require specific equipment (only available in certain rooms)

Implementation Requirements:

- Implement a CSP solver using:
 - Backtracking
 - **Forward Checking**
- Maintain:
 - Dynamic domain pruning after each assignment

Expected Output:

- Valid room allocation:

$M1 \rightarrow R2$

$M2 \rightarrow R1$

$M3 \rightarrow R3$

- Domain reduction logs after each assignment:

After assigning $M1 \rightarrow R2$, R2 removed from domain of M2

Task 8: CSP Formulation for Sudoku (9×9 Grid)

Formulate Sudoku as a CSP without implementing a full solver.

Requirements:

- Define:
 - Variables: one per cell
 - Domains: {1–9}
- Constraints:
 - All-different for:
 - Each row
 - Each column
 - Each 3×3 subgrid
- Represent constraints formally:
 - Using mathematical notation **AND**
 - Using programmatic structures

Expected Output:

- Formal CSP definition
- Data structures representing constraints